High-Level Synthesis of Hardware Accelerators using Bambu

Processor Design

Alba Soldevilla González

March 8, 2026

UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

Contents

1 Project Topic and Objectives

For this project, I have been assigned the task of converting software into hardware designs using **High-Level Synthesis (HLS)** techniques and different hardware design tools. The main goal is to translate C code into synthesizable Hardware Description Language (HDL) implementations such as VHDL and Verilog.

Once the conversion is completed, the generated hardware designs will be synthesized for **Field Programmable Gate Arrays (FPGAs)**. The resulting implementations will then be evaluated using a set of benchmarks in order to analyze their efficiency. Additionally, the results obtained with different tools will be compared to assess the quality of the generated hardware designs.

The overall workflow of the project will be as follows:

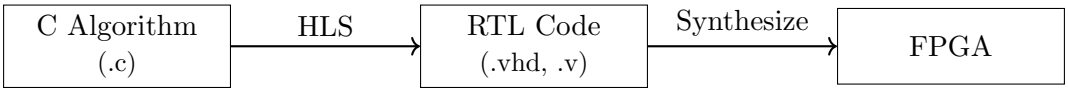

Figure 1: Simplified workflow of the hardware generation process.

For this first report, the focus will be on the High-Level Synthesis stage. In particular, this report demonstrates how C code can be translated into Verilog and VHDL and how the generated hardware can be verified through simulation using testbenches.

2 Tools

To perform the High-Level Synthesis process, two different tools will be used: **Bambu** and MATLAB. While my teammate will focus on MATLAB-based exploration, this work focuses on the use of Bambu for hardware generation.

On the other hand, the generated hardware designs will be synthesized and simulated using **AMD Vivado**. AMD Vivado is the design suite used for the development and synthesis of hardware designs targeting AMD FPGAs and adaptive SoCs. It provides several functionalities, including design entry, synthesis, placement and routing, verification, and simulation.

The workflow presented in Figure 1 can be updated as shown in Figure 2.

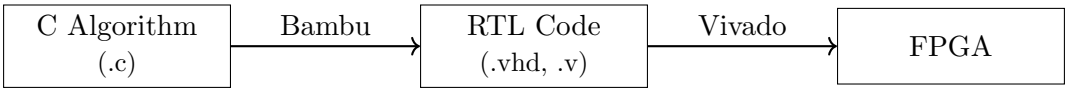

Figure 2: Toolchain used in this project.

At this stage, the focus is on learning how to use Bambu.

2.1 Bambu

Bambu is an open-source framework aimed at assisting designers during the High-Level Synthesis of complex applications, supporting most of the C constructs (e.g., function calls and module sharing, pointer arithmetic and dynamic resolution of memory accesses, accesses to arrays and structures, parameter passing either by reference or copy, ...).

Bambu receives as input a behavioral description of the specification, written in C/C++ language, and generates the HDL description of the corresponding RTL implementation as output, which is compatible with commercial RTL synthesis tools, along with a testbench for the simulation and validation of the behavior. Bambu is designed in an extremely modular way, implementing the different tasks of the HLS process, and specific algorithms, in distinct C++ classes which work on different IRs depending on the synthesis stage.

To better understand its usage, the Bambu tutorial provided on the official website will be followed and explained in the following section to illustrate its main functionalities.

3 Bambu Tutorial

3.1 Introduction and Preliminaries

For this tutorial, the PandA project repository was downloaded from `https://github.com/ferrandi/PandA-bambu`. The instructions provided on the official website (`https://panda.deib.polimi.it/?page_id=88`) were followed in order to install Bambu properly.

However, due to the Linux distribution used (Kubuntu 24.04.4 LTS x86_64), some of the required packages could not be installed using `apt`. Newer Linux distributions may not provide some of the dependencies required by Bambu in their repositories.

Nevertheless, the PandA-bambu repository provides a Docker Compose configuration file (located in `PandA-bambu/.devcontainer`) which builds a container with an Ubuntu 18.04.6 LTS environment. Therefore, in this work Bambu is executed inside this Docker container environment.

Once the environment is correctly configured, the installation can be verified by executing the following command:

```
$ bambu --version
```

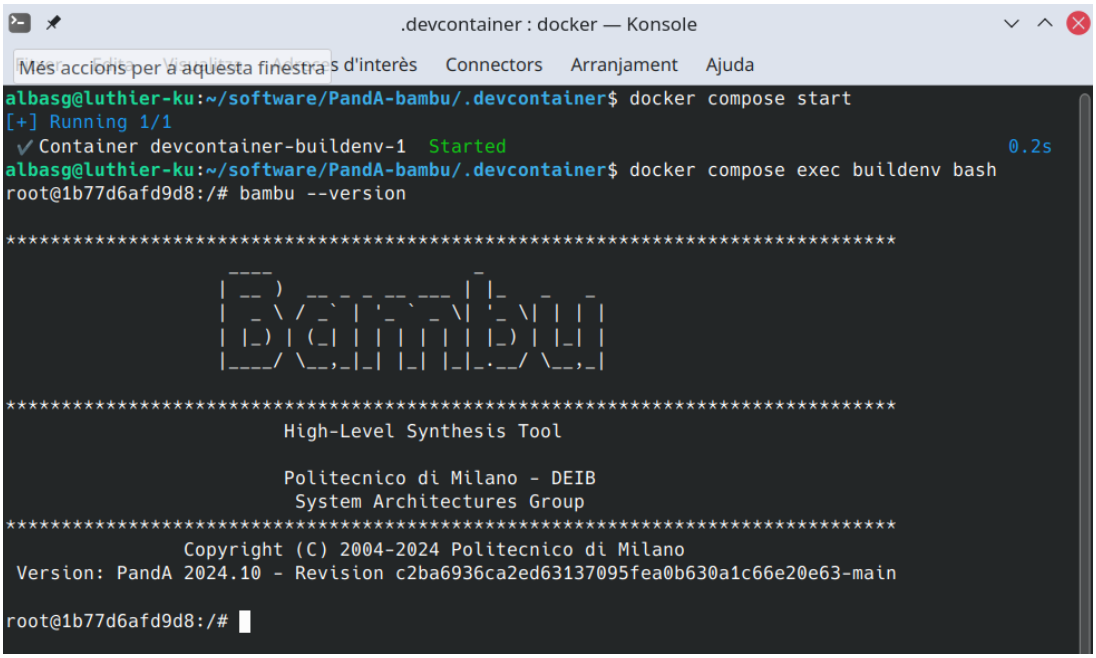

Figure 3: Bambu version verification.

3.2 Basic Usage - Generation of a Verilog design from C sources

Following the tutorial instructions, the corresponding directory was accessed and a Verilog file was generated using the following command:

```
$ bambu ../spec.c
```

This operation launches the default High-Level Synthesis (HLS) flow: a Verilog design will be generated from the `../spec.c` source file, where `spec.c` is the C code we want to convert.

Several files are generated as a result:

```
root@1b77d6afd9d8:/mirror/examples/crc/tutorial# ls -la
total 212
drwxr-xr-x 3 root root   4096 Mar  5 19:09 .
drwxrwxr-x 3 1000 1000   4096 Mar  5 19:09 ..
drwxr-xr-x 3 root root   4096 Mar  5 19:09 HLS_output
-rw-r--r-- 1 root root     17 Mar  5 19:09 array_ref_35512.mem
-rw-r--r-- 1 root root   4352 Mar  5 19:09 array_ref_35544.mem
-rw-r--r-- 1 root root   2304 Mar  5 19:09 array_ref_35602.mem
-rw-r--r-- 1 root root    144 Mar  5 19:09 array_ref_35630.mem
-rw-r--r-- 1 root root    396 Mar  5 19:09 array_ref_35747.mem
-rwxr-xr-x 1 root root    400 Mar  5 19:09 synthesize_Synthesis_main.sh
-rw-r--r-- 1 root root 174226 Mar  5 19:09 top.v
```

Figure 4: Generated files by executing Bambu.

Among these files we can find:

- `top.v`, the Verilog design generated from the C file.
- `synthesize_Synthesis_main.sh`, a shell script which can be used from the command line to launch the vendor tools for synthesis.
- `HLS_output`, directory where Bambu places different results. It contains Synthesis subdirectory with some empty subdirectories (input and output), a `vivado.tcl` script to launch the synthesis and a `main.sdc` file containing design constraints.
- `*.mem` files contain data for the initialization of the memories.

```
root@1b77d6afd9d8:/mirror/examples/crc/tutorial# ls -la HLS_output/Synthesis/vivado_flow_0/
total 28
drwxr-xr-x 4 root root 4096 Mar  5 19:11 .
drwxr-xr-x 5 root root 4096 Mar  5 19:11 ..
-rw-r--r-- 1 root root   54 Mar  5 19:11 icrc.sdc
drwxr-xr-x 2 root root 4096 Mar  5 19:11 input
drwxr-xr-x 2 root root 4096 Mar  5 19:11 output
-rw-r--r-- 1 root root 5524 Mar  5 19:11 vivado.tcl
```

Figure 5: Synthesis files generated by Bambu.

The High-Level Synthesis flow can be limited to a subset of functions present in the C file. For example, in `spec.c` file, the `main()` is only calling `icrc()` multiple times to test it.

```
$ bambu ../spec.c --top-fname=icrc
```

This will generate a Verilog file for the `icrc()` function in the current directory, as well as another subdirectory inside `HLS_output/Synthesis/` to handle the corresponding synthesis flow. A `synthesize_Synthesis_icrc.sh` shell script for synthesis will also be generated.

```
root@1b77d6afd9d8:/mirror/examples/crc/tutorial# ls -la
total 436
drwxr-xr-x 3 root root   4096 Mar  5 19:11 .
drwxrwxr-x 3 1000 1000   4096 Mar  5 19:09 ..
drwxr-xr-x 3 root root   4096 Mar  5 19:09 HLS_output
-rw-r--r-- 1 root root     17 Mar  5 19:09 array_ref_35512.mem
-rw-r--r-- 1 root root   4352 Mar  5 19:09 array_ref_35544.mem
-rw-r--r-- 1 root root     17 Mar  5 19:11 array_ref_35598.mem
-rw-r--r-- 1 root root   2304 Mar  5 19:09 array_ref_35602.mem
-rw-r--r-- 1 root root   4352 Mar  5 19:11 array_ref_35630.mem
-rw-r--r-- 1 root root   2304 Mar  5 19:11 array_ref_35667.mem
-rw-r--r-- 1 root root    144 Mar  5 19:11 array_ref_35699.mem
-rw-r--r-- 1 root root    396 Mar  5 19:09 array_ref_35747.mem
-rw-r--r-- 1 root root 205676 Mar  5 19:11 icrc.v
-rwxr-xr-x 1 root root    402 Mar  5 19:11 synthesize_Synthesis_icrc.sh
-rwxr-xr-x 1 root root    400 Mar  5 19:09 synthesize_Synthesis_main.sh
-rw-r--r-- 1 root root 174226 Mar  5 19:09 top.v
```

Figure 6: Synthesis files generated for the `icrc()` function within the main.

3.3 Simulation

Before actually starting the synthesis, it is possible to simulate the generated design. The C source code is compiled and executed with randomly generated inputs or with user-provided inputs. The execution of the C program returns a result which is compared with the result produced by the simulated Verilog: if they match the co-simulation succeeds, otherwise it fails and the difference is reported to the user.

```
1    $ bambu ../spec.c --simulate --simulator=VERILATOR
```

In general, one should provide input values for the simulation. If they are not specified, Bambu generates them randomly and saves them in a `test.xml` file in the current working directory. In this example, given that `main()` has no arguments, the `test.xml` file is an empty stub, like this:

```
root@1b77d6afd9d8:/mirror/examples/crc/tutorial# cat test.xml
<?xml version="1.0"?>
<function>
  <testbench/>
</function>
```

Figure 7: `test.xml` generated by Bambu for testing the simulation of `spec.c`

Bambu also generates a short report about the simulation cycles which contains information about how many times the design was simulated and how many cycles the simulation took to terminate. In my case:

```
Total number of flip-flops in function main: 32
Total cycles            : 2812 cycles
Number of executions    : 1
Average execution       : 2812 cycles
```

Figure 8: Report generated for testing the simulation of `spec.c`

To co-simulate the `icrc()` function, the test inputs cannot be generated randomly, because the

parameter unsigned char *lin is a pointer, and the `icrc()` function assumes that it points to an array of unsigned chars whose length is expressed by the parameter unsigned int len. For example, for `icrc()`, the `../test_icrc.xml` file (already provided) can be used.

Moreover a `results.txt` file was created during the execution in my working directory. This text file contains a detailed report of the number of cycles for each call. The tutorial documentation indicates that the `results.txt` file should contain one line per execution in the following format: `<success_flag> <cycles>`. However, when running the experiment with the current version of Bambu (PandA 2024.10), the generated `results.txt` had a different format:

```
root@1b77d6afd9d8:/mirror/examples/crc/tutorial# cat results.txt
7|5631,
0
```

Figure 9: Content from the generated `results.txt` file

Since this format did not match the one described in the tutorial, the generated simulation testbench was inspected to understand how the file was produced and I found that the simulation testbench records start and end **simulation timestamps**, from which the number of cycles can be derived using the clock period.

Therefore, the file represents a single execution:

- Execution: start = 7, end = 5631

Since the testbench defines `CLOCK_PERIOD = 2.0`, the number of cycles is obtained as:

$$\text{cycles} = \frac{\text{end\_time} - \text{start\_time}}{\text{CLOCK\_PERIOD}}$$

This gives 2812 cycles which matches the value reported by Bambu during the simulation.

The second line of `results.txt` indicates whether a mismatch occurred between the C and RTL models, where 0 indicates that no mismatches were detected and 1 indicates an error.

3.4 A complete example

This example aims to build a fixed-point binary-to-decimal converter. For the binary representation, an unsigned Q16.16 fixed-point format is used. The first 8 switches on the left in the picture are used for the integer bits, while the 8 switches on the right are used for the fractional bits.

3.4.1 Mapping C functions to hand-written HDL modules

Bambu invocation:

```
1    $ bambu ../led_example.c --top-fname=led_example ../IPs.xml
```

With respect to the previous examples, the significant change is that the file `../IPs.xml` is passed to Bambu along with the C source code. `../IPs.xml` contains a list of cells, in the form `<cell>...</cell>`. Each of them represents the mapping of a C function onto a Verilog module. Every cell also contains some information on the Verilog module, that will be used for the interconnection with other modules in the design and with external I/O ports during High-Level Synthesis.

The command used performs HLS on `led_example.c`, specifying to Bambu the Verilog binding, but the information provided to the tool is not enough to generate the final accelerator. Indeed,

in the generated `led_example.v` there are some instances of Verilog modules that are used but are not defined. The HLS flow completes successfully, but the generated design cannot be synthesized because the user-defined Verilog modules are not present. So, the command line has to be extended in this way:

```
$ bambu ../led_example.c ../IPs.xml --top-fname=led_example --file-
    input-data=../leds_ctrl.v,../sw_ctrl.v,../btn_ctrl.v,../
    sevensegments_ctrl.v
```

Here the argument `--file-input-data=../leds_ctrl.v, ../sw_ctrl.v,...` tells Bambu where to find the definitions of the Verilog modules. The given Verilog files will be copied in the working directory and the synthesis script will be set up to use them.

3.4.2 Simulation with hand-written HDL modules

When running simulations, Bambu automatically generates the testbench starting from the original C specification used for HLS. However, in this example some modules are hand-written in Verilog and are not generated through HLS.

Even if the location of the custom Verilog modules is provided, co-simulation cannot be executed because Bambu requires a complete C specification. Therefore, a reference C implementation must also be provided for each custom Verilog module so that Bambu can generate a golden reference for the co-simulation.

To include these reference implementations, Bambu can be executed as follows:

```
$ bambu ../led_example.c ../IPs.xml --top-fname=led_example --simulate
   --simulator=VERILATOR --file-input-data=../leds_ctrl.v,../sw_ctrl.v
   ,../btn_ctrl.v,../sevensegments_ctrl.v --C-no-parse=../leds_ctrl.c
   ,../sw_ctrl.c,../btn_ctrl.c,../sevensegments_ctrl.c
```

The option `--C-no-parse` adds these C files only for the co-simulation executable. They are not used for HLS, since the corresponding function calls are mapped to the user-defined Verilog modules described in `IPs.xml`.

```
  Total cycles             : 36 cycles
  Number of executions     : 1
  Average execution        : 36 cycles
root@1b77d6afd9d8:/mirror/examples/led_example/tutorial# ls
HLS_output           led_example.v         simulate_led_example.sh
array_ref_35500.mem  leds_ctrl.v           sw_ctrl.v
bambu_results_0.xml  results.txt           synthesize_Synthesis_led_example.sh
btn_ctrl.v           sevensegments_ctrl.v  test.xml
root@1b77d6afd9d8:/mirror/examples/led_example/tutorial# cat results.txt
7|79,
0
```

Figure 10: Content from the generated `results.txt` file

The mappings have been done perfectly and the results file was also generated along the short report about its simulation.

3.4.3 Generation of the bitstream with Bambu

For the last part of the tutorial, Vivado is required in order to generate the FPGA bitstream. Since the Bambu tutorial has already been completed and the generated hardware design is

available, Vivado can be used independently from Bambu; therefore, it is not necessary to run Bambu again to use Vivado.

Because the installation of Vivado requires a large amount of disk space, it was installed locally instead of inside the Docker container. Consequently, the next step is to learn how to use Vivado locally by transferring the files generated by Bambu from the Docker container to the host system.

4 Additional Example: Simple Adder

To further validate the HLS workflow, an additional example was implemented independently from the tutorial.

I decided to test the process by generating RTL code from a simple function of my own choice (different from the one provided in the tutorial). I wrote a simple addition function in C:

Listing 1: `sum` function in C used as input for the HLS process.

```c
int sum(int a, int b){
    return a + b;
}
```

4.1 Hardware Generation

The design was synthesized using the following command:

```
    $ bambu sum.c --top-fname=sum
```

The option `--top-fname=sum` was used because the file `sum.c` does not contain a `main()` function but only the function itself. Therefore, Bambu needs to know explicitly which function should be used as the top-level module for hardware generation.

```
root@1b77d6afd9d8:/workspace# ls
HLS_output  build  sum.c  sum.v  synthesize_Synthesis_sum.sh
root@1b77d6afd9d8:/workspace# ls HLS_output/Synthesis/vivado_flow/
input  output  sum.sdc  vivado.tcl
```

Figure 11: Files generated from the `sum.c` source code.

As shown in Figure 11, the RTL file (`sum.v`) was successfully generated, as expected.

```
53  module plus_expr_FU(in1,
54    in2,
55    out1);
56    parameter BITSIZE_in1=1,
57      BITSIZE_in2=1,
58      BITSIZE_out1=1;
59    // IN
60    input signed [BITSIZE_in1-1:0] in1;
61    input signed [BITSIZE_in2-1:0] in2;
62    // OUT
63    output signed [BITSIZE_out1-1:0] out1;
64    assign out1 = in1 + in2;
65  endmodule
```

Figure 12: Excerpt of the Verilog code `sum.v` showing the functional unit that implements the addition operation.

4.2 Testbench Simulation

As stated in the Bambu tutorial, the following command can be used to run a simulation:

```
$ bambu sum.c --top-fname=sum --simulate --simulator=VERILATOR
```

Since no `test.xml` file was provided, Bambu automatically generated one with random input parameters. A short simulation report was printed in the terminal and a `results.txt` file was generated in the working directory.

```
Warning: XML file "test.xml" cannot be opened, creating a stub with random values
  Total cycles            : 1 cycles
  Number of executions    : 1
  Average execution       : 1 cycles
root@1b77d6afd9d8:/workspace# cat test.xml
<?xml version="1.0"?>
<function>
  <testbench a="18" b="7"/>
</function>
root@1b77d6afd9d8:/workspace# ls
HLS_output          build       simulate_sum.sh  sum.v                       test.xml
bambu_results_0.xml  results.txt  sum.c            synthesize_Synthesis_sum.sh
root@1b77d6afd9d8:/workspace# cat results.txt
7|9,
0
```

Figure 13: Simulation report of the `sum` function.

The `results.txt` file shown in Figure 13 indicates that no error occurred during the simulation. For the test defined in `test.xml`, using the values $\mathbf{a = 18}$ and $\mathbf{b = 7}$, both the C model (`sum.c`) and the generated RTL implementation (`sum.v`) produced the same result. Therefore, no mismatch was detected between the software model and the synthesized hardware.

4.3 Simulation with Icarus Verilog

Another way to test the generated `sum.v` module is to create a testbench (`tb.v`) and simulate the RTL implementation using a different simulator, such as Icarus Verilog. The resulting waveform signals can then be inspected using GTKWave. All these tools had been installed previously.

The Verilog testbench used to simulate the generated RTL with Icarus Verilog is shown in Listing 2.

Listing 2: Simple testbench for the `sum` module used with Icarus Verilog.

```
`timescale 1ns/1ps

module tb;

  reg [31:0] a;
  reg [31:0] b;
  wire [31:0] return_port;

  // Device Under Test (DUT)
  sum uut (
    .a(a),
    .b(b),
    .return_port(return_port)
  );

  initial begin
    // Generate waveform file for GTKWave
```

```
18      $dumpfile("sum.vcd");
19      $dumpvars(0, tb);
20
21      // Test values taken from test.xml
22      a = 32'd18;
23      b = 32'd7;
24
25      #10;
26
27      $display("a=%0d b=%0d -> result=%0d", a, b, return_port);
28
29      if (return_port == 32'd25)
30        $display("TEST PASSED");
31      else
32        $display("TEST FAILED");
33
34      #10;
35      $finish;
36    end
37
38  endmodule
```

The simulation was executed using Icarus Verilog, as shown in Figure 14.

```
root@1b77d6afd9d8:/workspace# iverilog -o sum_tb tb.v sum.v
root@1b77d6afd9d8:/workspace# vvp sum_tb
VCD info: dumpfile sum.vcd opened for output.
a=18 b=7 -> result=25
TEST PASSED
```

Figure 14: Icarus Verilog simulator output.

The `iverilog` tool compiles the Verilog sources, including the testbench (`tb.v`) and the synthesized module (`sum.v`), producing a simulation executable named `sum_tb`. The `vvp` command then executes the simulation and generates a waveform file (`sum.vcd`).

Since the simulation was executed inside a Docker container without graphical display support, the `sum.vcd` file was copied to the host system and visualized locally using GTKWave, as shown in Figure 15.

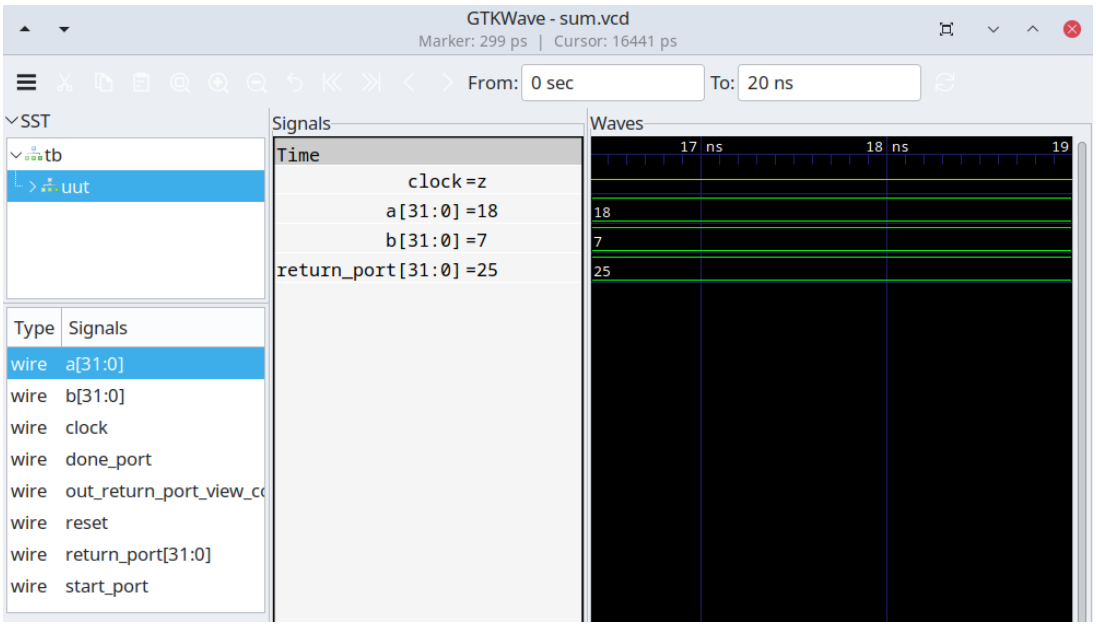

Figure 15: Waveforms generated by the `sum` testbench visualized with GTKWave.

5 Current Status

At the current stage of the project, the Bambu toolchain has been successfully installed and tested through the official tutorial examples. RTL generation from C code was validated, and co-simulation was used to verify consistency between the original software model and the generated hardware.

In addition, an independent simple adder example was developed to further confirm the correctness of the workflow. These preliminary results validate the initial HLS flow and provide a solid basis for the next stages of the project.

6 Future Work

The next stage of the project will focus on synthesizing the generated hardware designs using AMD Vivado in order to obtain implementation metrics such as resource utilization, maximum operating frequency, and latency.

Additionally, more complex algorithms from the selected benchmark suite will be explored and translated into hardware using Bambu. These implementations will allow the evaluation of the efficiency of the generated RTL designs.

Finally, the results obtained with Bambu will be compared with those produced using alternative tools, such as MATLAB-based workflows, in order to assess the quality and performance of the different High-Level Synthesis approaches.

References

[1] PandA Project. *PandA Framework for High-Level Synthesis*. Available at: `https://panda.deib.polimi.it/`

[2] Ferrandi, F. et al. *PandA-Bambu High-Level Synthesis Tool (GitHub Repository)*. Available at: `https://github.com/ferrandi/PandA-bambu`

[3] AMD. *Vivado Design Suite*. Available at: `https://www.amd.com/es/products/software/adaptive-socs-and-fpgas/vivado.html`